A Seed That Grows by Testing Its Own Changes: Persistent State Improvement Across Five Areas and Three Development Seeds

Maureese Williams

October 11, 2026

Abstract

This paper studies a narrow form of self-improvement: can one persistent agent improve the state on which a fixed learning procedure operates, test each proposed change before keeping it, and carry the accepted state into settings with different structure? The state is the agent's accumulated experience, the models that guide its search, and the primitives from which it builds candidate solutions; the procedures that fit models, invent primitives and accept changes do not change. The agent solves rule-inference tasks over characters by searching a catalog of candidate predicates under a scan budget, and it can fail for two reasons: the right predicate exists but lies outside the budgeted portion of the catalog, or its current primitives cannot express it. Learned search guidance addresses the first, invention of primitives the second, and a pruning step removes inventions that never pay. A change is retained only when it solves more tasks than the current state on held-out worlds the agent never trained on, and a change to the shared primitives must also leave previously learned settings within a stated tolerance; that second condition was added after a change accepted in one setting cut a learned setting's score by 10%. On held-out sets never used for acceptance, the main agent's solve rate in family-structured worlds rose from 17.1% to 28.8% over twelve experience cycles, and it rose in four further settings with different hidden structure; two agents started from other random seeds rose from 17.9% to 26.1% and from 15.0% to 28.8% in three cycles of the first setting, and in two further settings each, with one exception where a shared change passed a cross-area check too small to catch a 12% loss. Learned search guidance was the most consistently accepted kind of change; invention paid off less often; untrained family combinations in the first setting ended 9% below their start. The development results were inspected as the design changed. A confirmation protocol frozen afterwards, with four pre-stated gates and a committed forecast, was run on sixteen sealed random seeds across eight versions: every acceptance stood on replay on all sixteen; the first version failed its cost gate, at six times the cost of living, the next two failed one audit clause each, the fourth passed all four gates on both of its seeds, with testing at about twice the cost of living, and a fifth, which added a label-free guard for the unguided half of the search budget, failed on one seed the growth and safety gates and on the other the cost gate, and was retired. The sixth and seventh kept the fourth's code and widened the schedule to six areas, three of them lived by no seed before: all 24 area scores rose on fresh held-out sets, by 23% to 542%, every pool rose, and the seventh, whose cost gate scales with the number of areas checked, passed all four gates on both seeds. An eighth added a seventh area of a different task form, string transformations in a small program language with three visible and six hidden examples per task, and passed all four gates on both of its seeds: the string area rose 27% and 32%, and both seeds invented the same six withheld positional primitives. The learning procedures are fixed throughout.

1 Introduction

An agent can improve its performance without improving the algorithm that teaches it. It can accumulate useful evidence, learn where to search, or acquire a new primitive that makes a previously inaccessible solution expressible. These are different forms of growth, and they can be carried forward or lost.

This research programme uses *recursive self-improvement* to mean a system producing a successor with a better improvement procedure; that is the definition used here, not a universally accepted one. The present paper studies a narrower question: can one agent improve the persistent state on which a fixed learning procedure operates? That state includes its accumulated experience, the models that guide its search, and the primitives from which it constructs solutions. Changes to this state influence subsequent problem solving and the experience available for later updates, but the update procedures themselves do not change. The contribution is the integration and evaluation of several state-improvement mechanisms within one persistent agent, with an acceptance test that decides which changes remain.

Retaining improvements is straightforward only when their effects remain local. In this system the primitives are shared across settings. An update that helps the current setting can therefore change the search space of a setting learned earlier. The experiments expose exactly this failure and motivate testing shared changes wherever they will be used.

The agent is called a *seed* because it is meant to be the thing that grows: a saved state plus fixed update procedures, run for successive cycles of experience. To avoid a clash with pseudorandom seeds, the paper says *seed agent* or *agent instance* for the artifact and *random seed* for identifiers such as 644101.

Antecedents. Each form of growth had been shown separately in this programme, in separate code. Within-life accumulation: an agent that keeps the atoms of whatever it submits, right or wrong, raised its own solve rate four to ten times across a hundred tasks at every scan budget from 128 to 4,096, and being told whether it was right added nothing. Learned search guidance: a grouping of primitives learned without labels from 1,600 worlds solved 6,993 and 8,077 more tasks over 18 budget cells than the same agent on a grouping from 400 worlds, on prespecified random seeds, audited. Expanded representational reach: in a string-manipulation language, a system that synthesised primitives from its failures solved 20 of 24 tasks where its fixed grammar solved 5, on two random seeds. None of these was recursive in the programme's sense; every attempt at a better child improver returned a null result. What they shared was a state that grew with experience and a learner that did not.

2 The task, and the words this paper uses

2.1 A task in this system

Each task asks the agent to infer an unknown Boolean rule over characters from a small set of visible examples. The agent searches a catalog of candidate rules and submits one that agrees with the visible examples. Hidden examples are reserved for evaluation. The search budget limits how many candidate rules the agent can inspect.

Here is one task from the implementation, drawn from a world whose hidden structure is described below. Characters are given by their code within a 256-character alphabet. The true rule is that bit 7 of the code is 0, that is, the code is below 128. Each visible example is a string

whose longest prefix of characters satisfying the rule is the answer; the first three of the eight visible examples are

answer prefix	rest of the string
119, 115, 48, 47, 121	152, 242, 205
23, 10, 101	168, 131, 144, 143, 137
61, 7, 118	240, 178, 187, 203

A candidate rule is consistent with an example when every character of the answer prefix satisfies it and the first character after the prefix does not. The agent's catalog at the start of this work has 82,758 candidate rules. Of these, 77 are consistent with all eight visible examples, and 17 of the 77 also reproduce the sixteen hidden examples. Scanning the catalog in its starting order with a budget of 1,024 rules, the agent meets exactly one consistent candidate, "code below 144", which is built from the single primitive `threshold:144` and its complement. It submits that rule. The rule fails a hidden example, so the task is not solved; the agent is not told this; and the primitive `threshold:144` enters its experience of this world as if it belonged there. The right rule, built from the primitive `bit:7:0`, was in the catalog all along but outside the 1,024 rules inspected. The agent's record of this submission notes that it found one consistent candidate in its window, which makes the submission a *confident pick* in the sense defined below; on average such picks are correct 90% of the time, and this one was not.

2.2 Two hierarchies: how problems are generated, and how experience is scheduled

Two organisations should be kept apart. On the generation side, an *area* is a distribution over worlds with a particular hidden structure of primitive co-occurrence; a *world* is one hidden set of 14 primitives, and every task of that world is drawn from a rule built only from them, so a world's tasks are related. On the schedule side, a *life* is one cycle of experience and update: the agent lives 48 worlds of one area, 100 tasks each, and only then proposes and tests changes to its state. The task above came from a world of the first area, drawn from the families *bit*, *divmod* and *gcd*.

2.3 Terms

Term	What the reader should understand by it
seed agent	A persistent agent consisting of saved state and fixed update procedures.
atom	An elementary Boolean predicate over characters, such as "bit 7 of the code is 0", used as a building block for candidate rules. There are 313 in 9 families named for their arithmetic operation.
language	The collection of atoms the agent currently holds; not a natural-language vocabulary.
hypothesis catalog	Every atom, every pair of atoms from different families under and, or and xor, and every complement, reduced to one row per distinct behaviour over the alphabet. Rebuilt from the language whenever it changes: 82,758 rows from the starting 260 atoms, 166,596 from all 313.
world	A hidden set of 14 atoms from which related tasks are generated.
area	A distribution over worlds with a particular co-occurrence structure.
life	One experience-collection and update cycle: 48 worlds, then proposals and tests.
earned experience	The atoms behind the agent's own submissions in a world, kept whether the submission was right or wrong.
confident pick	A submission made when exactly one consistent candidate lay in the scanned window. Not uniqueness in the whole catalog, and not certainty.
world and self model	An area's model of which atoms occur together and how often the agent finds each; it guides search.
carried state	Everything the agent keeps between lives.
prove	Run an empirical acceptance comparison between the carried state and a candidate on held-out worlds; not a mathematical proof. The paper says *test* and *accept*.

3 Why the agent fails, and what it does about it

The agent can fail for two different reasons. A suitable predicate may already exist in its catalog but lie outside the portion inspected under the search budget, as in the task above. Alternatively, the current language may lack the primitives needed to construct it; the starting language omits two of the nine families, so a third of the rules in some worlds cannot be expressed at all. Learning a model can help with the first problem by changing which hypotheses are searched first. Extending the language can help with the second by adding new building blocks. Pruning addresses a related cost: added primitives enlarge the catalog and can dilute the budgeted search without improving performance. Two qualifications prevent misreadings. A task left without any consistent candidate is not proof that a primitive is missing; it means the budgeted search found none. And invention happens within a fixed expression grammar, not by unrestricted construction of new algorithms.

3.1 What persists

The carried state is the language; the experience, one record per world lived, with the area and the atoms behind the agent's confident picks and behind all its picks, by atom name so that the records survive a change of language; per area, a world and self model and a flag saying whether to build a search prior from that area's experience; the invented atoms and the life they entered; a ledger of every candidate change with its test; and the resource spent on living, testing and inventing. The language, the experience and the ledger are shared across areas; the model and the prior belong to an area.

3.2 The operations

Live collects evidence while solving related tasks. The agent draws fresh worlds of an area and lives each as a stream of 100 tasks at the scan budget. Within a world, the atoms of each submission reorder the catalog for the next task, which is the within-life accumulation of the antecedents. Two carried objects reorder it further. The area's model, when it exists, turns the atoms earned so far in the current world into beliefs about which groups of atoms the world uses, and those beliefs promote the rows those groups build. The area's prior, when it exists, starts the scan from an order given by how often atoms co-occurred in the agent's own earlier worlds there, with half of each task's budget reserved for the order it would use with no prior at all. With neither, the agent scans in catalog order. The agent receives no correctness feedback on any submission.

Reflect uses accumulated evidence to improve future search guidance. The world and self model describes both which primitives tend to occur together and how often the agent discovers particular primitives; its purpose is to use the agent's own submissions to guide subsequent search. Latent: a group label per atom, and which groups each world used. Observed: the confident picks of every world of the area. Fitted, by expectation-maximization: the labels, how often each set of groups occurs, and for each atom a *discovery rate*, how often the agent finds the atom when it is present in a world, and an *import rate*, how often the agent earns the atom through a wrong pick in a world where it is absent. An atom whose discovery rate falls below 0.10 is filed in no group: the agent meets it only through its own mistakes. Downstream, during a new world the collected atoms update the agent's beliefs about which groups are present, and those beliefs promote the corresponding hypotheses in the search order. The model's size, 6 to 8 groups and 2 to 4 groups per world, is chosen by the likelihood of the agent's last fifth of worlds in that area. The refit model and a prior from the area's experience form one candidate change. The fuller account of the model is in the companion paper.

Extend proposes additional primitives from uncovered tasks. For the third of the life's worlds with the most tasks left without any consistent candidate, and only worlds with at least 15 such tasks, the agent enumerates expressions of depth one from the ordinal grammar and tests each against the constraints of the uncovered tasks, alone, complemented, or joined by and, or or xor to one atom it earned in that world. A candidate atom is proposed only if it fits at least two uncovered tasks: an atom a world is built from recurs across the world's rules, and a chance fit does not. The grammar yields 3,456 candidates distinct up to complement, searched smallest first; at most 40 atoms are proposed per life.

Prune proposes removing unused inventions. An invented atom is proposed for removal two or more lives after it entered if it has never appeared in a confident pick, or if every area model that has seen it rates it as found only through wrong picks.

Prove determines whether a change deserves to persist. After a life the agent proposes all three kinds of change at once and tests the combination; if the combination fails, it tests each part alone. A test runs the current state and the candidate on a held-out set of the area's worlds, drawn for that life and never lived. Under the rule in force at the end of this work, the candidate is accepted only when its gain over the current state, paired over the held-out worlds, has a 95% bootstrap interval above zero; a candidate clearly behind on the first half of the set is stopped there. A candidate that changes the language is also run on a smaller held-out set of each other area the agent has lived in and is rejected if its total there falls more than 2% below the current

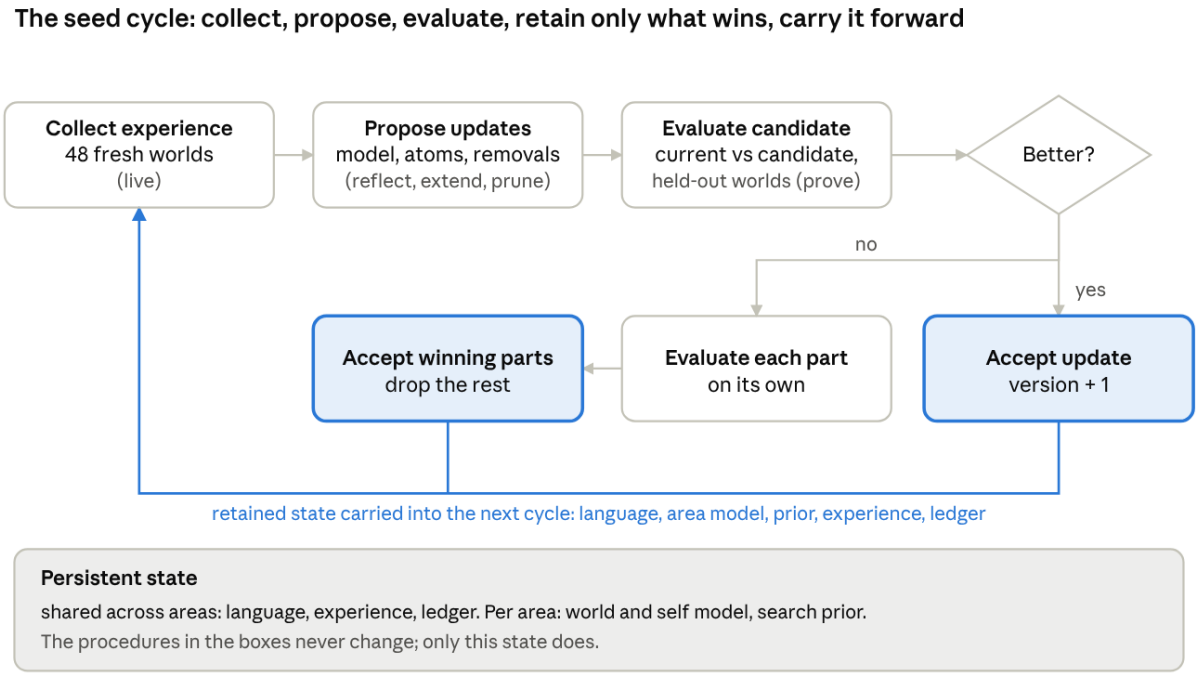


Figure 1: One cycle of the seed agent. It collects experience in 48 fresh worlds, proposes updates of three kinds, evaluates the combined candidate against the current state on held-out worlds drawn for that cycle, accepts it if it wins and otherwise evaluates each part on its own, and carries the retained state, language, area model, prior, experience and ledger, into the next cycle. The language, experience and ledger are shared across areas; the model and prior belong to an area. Implementation names in parentheses.

state's, a relative decrease on that set. Every held-out lifetime is charged to the agent's resource. The agent therefore receives aggregate performance feedback when selecting updates, though never per-submission feedback while collecting experience. Section 5 gives the history of this rule, which was revised twice during development, by the experimenter and not by the agent.

Figure 1 shows the cycle and what travels from one cycle to the next. The procedures in the boxes never change; the state on the return path does.

4 Areas

Mechanism worlds (`mech`). A world draws three of seven families and its 14 atoms from their pooled atoms. The starting language lacks the two remaining families, popcount and xormod, so a world that uses one of them holds rules the agent cannot express. A fifth of the viable family triples are withheld from the agent's lives as *untrained combinations*. Lives draw three quarters of their worlds from trained triples and a quarter from triples with an unfamiliar family; held-out sets hold equal numbers of trained, untrained and unfamiliar worlds.

Drawn partitions (`drawn7`, `drawn11`, `drawn13`). Every atom of every family is assigned at random to one of a few groups; a world picks some groups and draws its 14 atoms from their union,

and each atom slips to a uniformly random atom with some probability. The partition cuts across families, so nothing learned about mechanism worlds applies except the language. The three areas differ in structure: 9 groups with 3 per world and slip 0.15; 12 groups with 4 per world and no slip; 6 groups with 2 per world and slip 0.15. The agent is not told how many groups a world uses.

Metric worlds (`metric777001`). Each atom has a behavioural descriptor, the normalised Fourier and Walsh spectra of its truth table over the alphabet. A sparse weighting over descriptor dimensions is drawn once and shown to nobody. A world is a centre atom together with the atoms nearest it under that weighting, fourteen in all, the centre included as the nearest to itself. A quarter of each life's worlds are centred on an atom of the two families the starting language lacks. Here the hidden structure is behavioural rather than syntactic.

5 Evidence boundaries, protocol and chronology

String tasks (`str7`). A second task form, added for the eighth confirmation version. A task is three visible and six hidden input-output pairs of a hidden program in the trapdoor string language: a program concatenates up to three rows, each the substring between two positions, with or without a case operation, or a constant. The positions (character offsets, and the start or end of the first, second or third run of letters, digits, upper-case or lower-case characters), the three case operations and six constants are the keys, 58 in all, and the 9,400 rows they build are the catalog. A world is 14 keys from three of seven key families and 100 tasks over programs on those keys alone, the mechanism structure transposed, with trained, untrained and unfamiliar pools; the twelve positions at upper-case and lower-case runs are withheld from the starting language. The agent's window is the first b rows of the catalog in the order its evidence, prior and model give; a task is solved by the first program within the window that reproduces the visible pairs, and counted only if it reproduces the hidden ones. Extend proposes keys the language lacks that complete programs for at least two uncovered tasks of a stuck world. The same state, operations and acceptance test apply; a language change is checked across areas of the same form only, since the two forms build their catalogs from disjoint keys.

What information is used where. Table 1 lists the evidence each stage reads and the decision it supports. The model-selection subset is part of accumulated experience, not a separate dataset; the final evaluation sets are separate and were never used to accept anything.

Table 1: Evidence and decisions by stage.

Stage	Evidence or measurement used	Decision it supports
Task experience	Visible examples, submissions, consistency counts, uncovered tasks	Search and experience collection; no per-submission correctness feedback
Model selection	Held-out likelihood on the last fifth of the area's accumulated worlds	Choice among model sizes
Update acceptance	Aggregate held-out solve counts, paired over worlds	Whether a proposed state change is retained
Final evaluation	Further held-out sets not used to accept updates	The reported gains

Schedule. A life is 48 worlds of 100 tasks at a budget of 1,024 scans per task. State is saved after every operation, so lives in different areas can be interleaved and a run resumed. The main agent, `s1` on random seed 644101, lived twelve lives in mechanism worlds, four in `drawn7`, four in `metric777001`, three in `drawn11` and two in `drawn13`, in that order except that mechanism lives 10 and 11 came last. Agent `s2` on random seed 643101 lived three mechanism lives, then two in `drawn7` and two in `metric777001`; agent `s3` on 645101 lived the same schedule.

Held-out sets. An acceptance test uses 24 worlds per pool of the area (72 for mechanism worlds, 24 for a single-pool area), and the cross-area check 12 per pool. From version 11 of `s1` onward every life draws its own held-out set; the first eight mechanism lives (versions 0 to 10) reused one fixed set, and the numbers read on it carry the optimism that reuse allows. The reported gains come from one further held-out set per area, drawn with a salt no test used, on which the carried state and the starting state (starting language, no model, no prior) are run once each.

The acceptance rule, by version. The rule was revised by the experimenter as results exposed its weaknesses (Table 2); the agent did not alter it. The ledger records which rule accepted each change.

Table 2: Which acceptance rule governed which versions of `s1`.

Versions (lives)	Rule
0 to 10 (mechanism lives 0 to 7)	Candidate accepted if it solves strictly more than the current state on one fixed held-out set; single-area test; one model for the one area.
11 to 22 (mechanism 8 to 9, `drawn7` 0 to 3, `metric` 0 to 3)	As above, but the held-out set is drawn fresh for each life and models and priors are kept per area.
23 to 25 (`drawn11` 0 to 2, `drawn13` 0 to 1, mechanism 10)	Combination tested first, then parts; a language change must also stay within 2% on every other area lived; groups per world chosen by held-out likelihood.
26 (mechanism 11)	Accepted only if the paired 95% bootstrap interval of the gain is above zero; tie-break a hash of row content; pruning also on the self model's verdict.
after 26	Half-set early stop, version snapshots for audit, held-out sets of at least 48 worlds and cross-area sets of at least 24 worlds; the rule of the confirmation runs.

What the agent never sees. A family label, a true world, which pool a world came from, or whether a submission was correct. Labels enter only in scoring, to say which pool a world came from and which family an invented atom reproduces.

Resource. The ledger charges living (worlds $\times$ 100 tasks $\times$ budget scans), every held-out lifetime of every test, and every invention test.

6 Results

Did performance improve? In ten of eleven agent-and-area pairs of the development runs, yes, and in all 68 pairs of the sealed confirmation seeds. At a fixed budget of 1,024 scans per task, the

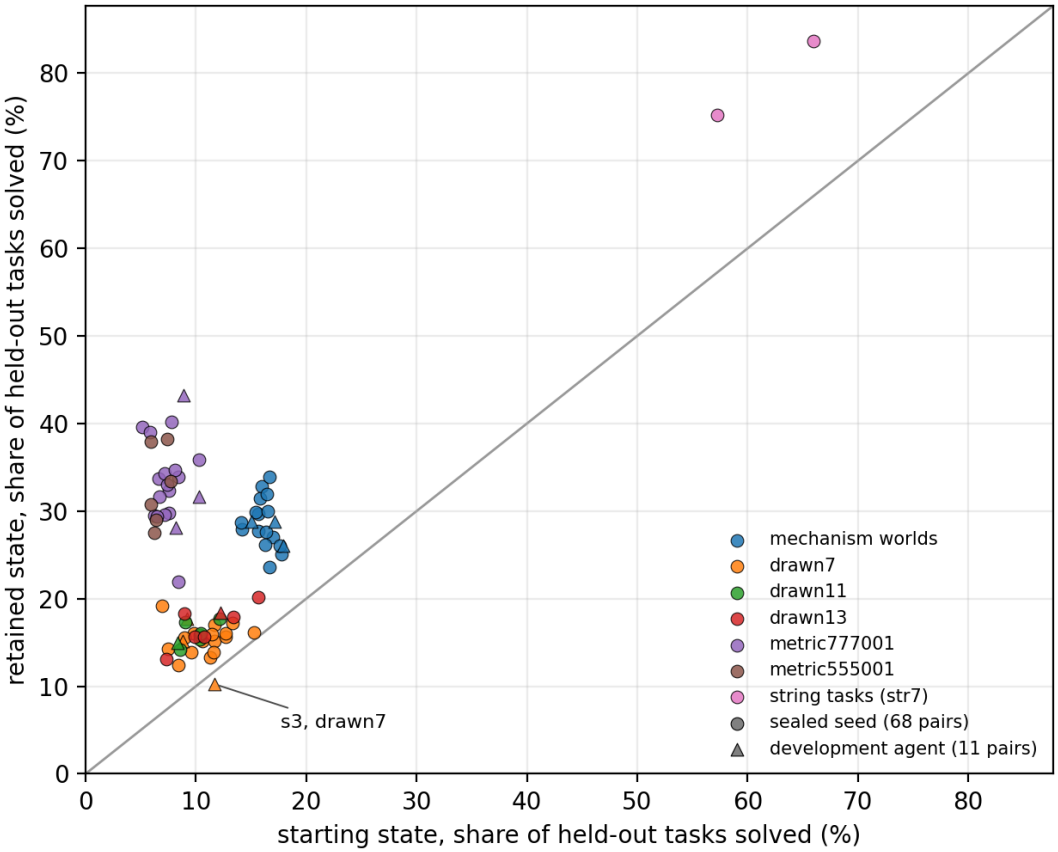


Figure 2: Share of held-out tasks solved at budget 1,024 on sets no acceptance test used: the starting state against the retained state, one point per agent instance and area, 79 pairs in all (the three development agents in eleven, the sixteen sealed seeds in 68). A point above the diagonal improved. Each point is one draw of 24 worlds per pool; the check recorded totals only, so no interval is attached (Section 9). One pair fell, `s3` in the drawn partition.

main agent's mechanism-world solve rate on held-out worlds never used for acceptance rose from 17.1% to 28.8% over twelve lives (1,232 to 2,072 of 7,200 tasks), with one qualification: trained family combinations rose from 26 to 49 tasks per world and worlds with an unfamiliar family from 5 to 19, but untrained family combinations fell from 20.5 to 18.6 (Section 8). Figure 2 carries the overview and Table 3 the counts. Five-area coverage is the main agent's; the two further agent instances cover mechanism worlds for three lives and the first drawn partition and metric worlds for two lives each. The exception is `s3` in the drawn partition, where nothing was accepted in two lives on 24-world held-out sets and the retained state reads 280 to 246 on the untouched set: the loss came from an extension of 40 atoms accepted in metric worlds, which passed its 12-world cross-area check on the drawn partition (124 to 148) and failed on the larger untouched set. The cross-area check is too small.

Which updates were useful? Model updates were the most consistently accepted kind of change: on the main agent a refit model was accepted on every mechanism life from 0 to 6 and on the first lives of every other area, and on `s2` and `s3` on all three mechanism lives. Language extensions were accepted less often, 6 of 24 proposed on the main agent, and every one proposed on `s2` and `s3` in

Table 3: Tasks solved on held-out sets never used by an acceptance test, 24 worlds per pool at budget 1,024, all under the content-stable tie-break.

Agent (random seed)	Area (lives)	Start	Retained	Tasks
s1 (644101)	mechanism worlds (12)	1,232	2,072	7,200
s1	drawn7, 9 groups of 3 (4)	211	364	2,400
s1	metric worlds (4)	427	2,076	4,800
s1	drawn11, 12 groups of 4 (3)	199	360	2,400
s1	drawn13, 6 groups of 2 (2)	294	442	2,400
s2 (643101)	mechanism worlds (3)	1,292	1,879	7,200
s2	drawn7 (2)	221	424	2,400
s2	metric worlds (2)	492	1,520	4,800
s3 (645101)	mechanism worlds (3)	1,081	2,074	7,200
s3	drawn7 (2)	280	246	2,400
s3	metric worlds (2)	394	1,349	4,800

mechanism worlds was rejected. Accepted removals, four on the main agent, took out 23, 26, 25 and 55 invented atoms. Over 26 versions the main agent accepted 16 models, 6 extensions and 4 removals and rejected 18 extensions, 6 models and 1 removal. These are acceptance outcomes, not a decomposition of the final gain: the rise on unfamiliar-family worlds, which the starting language cannot express a third of, needs invention, but the measurements do not separate invention's share from improved search.

What was invented? At version 26 the language holds 111 invented atoms: 26 reproduce true popcount atoms and 29 true xormod atoms, the two withheld families, and 56 are behaviourally new. Invented atoms entered only through an accepted test. In metric worlds a model fitted from the first 48 worlds was alone worth 975 tasks on that life's held-out set, and in the drawn areas the agent chose four groups per world for the 12-group area, its true structure, and four for the 6-group area, where the truth is two.

What failed? Two things, each of which changed the acceptance rule. A change accepted in one area damaged a learned area (Section 7). And the grown language, through atoms the agent's own model rated as never found, diluted the budgeted search on untrained combinations (Section 8). A third weakness was in the rule itself: on life 10 a model was accepted for a gain of 24 tasks on 72 worlds that read 82 lower on the untouched set, which is why acceptance now requires an interval above zero.

What did it cost? By version 26 the main agent had spent 122.9 million scans living, 302.3 million testing, and 831,738 invention tests. Testing cost two and a half times living, because each life tested up to four candidates on 72 worlds against a freshly scored current state and the cross-area checks added to that. The per-life held-out totals the agent decided by are in the ledger; they lie within a few percent of the untouched-set numbers except for the life-10 case above.

Confirmation on sealed random seeds. After the design was frozen, with the hashes of its source files, a protocol with four pre-stated gates and a committed forecast was run on random seeds never used before: four mechanism lives, three in `drawn7`, three in metric worlds, two more

mechanism lives, then the fresh check of Table 3 and a replay of every accepted change. The gates were growth of at least 15% in every area, no pool of any area below 90% of its start, testing at most 2.5 times the cost of living, and every accepted change replaying exactly from its own held-out set. The protocol went through four versions, each frozen with new hashes and run on two new sealed seeds, and each revision came from a failed gate (Table 4). Version 1 passed growth, safety and replay on both seeds and failed cost on both, at 5.96 and 5.88 times living; the arithmetic is that a fresh held-out set every life makes the re-score of the current state alone 1.5 times a 48-world life in the mechanism area, with three candidates per life on top. Version 2 changed the schedule's economics and not the acceptance rule: lives of 96 worlds, 48-world sets in every area, a held-out set kept until something is accepted on it, and a stop at the half set when the mean gain is not positive. It passed cost on both seeds, at 1.89 and 1.98, and failed on one of them the clause that each acceptance has its own set, because two parts of one life were accepted on the same set. Version 3 drew a fresh set after any acceptance within a life and failed the replay once on each seed, by 47 and 27 tasks in the recorded baseline: the current state's cached score on a reused set had gone stale as the area's experience, from which its search prior is built, grew. Every acceptance stood on replay. Version 4 keys that cache on the experience count as well as the version and passed all four gates on both of its seeds, at 2.32 and 2.04 times living. Across the eight seeds every area rose by at least 17% and every pool of every area rose except one, which ended at 94% of its start. The cross-area check rejected shared changes on most seeds for losses on the mechanism area that the proposing area's own test had accepted. Version 5 added the guard the development results had asked for: a candidate that adds atoms to the shared language is also scored with the model and prior switched off, on half the set, and rejected if its unguided search falls more than 10% below the carried language's. On seed 919108 the guard never fired, three extensions passed at unguided losses of 2% to 9% each, and the untrained pool ended at 89% of its start while the drawn partition grew 6%; on seed 919109 the guard fired three times, every pool rose, and testing cost 2.53 times living. A per-candidate tolerance does not bound a cumulative drift, and the guard was retired; version 4 is the confirmed protocol for three areas. Versions 6 and 7 kept version 4's code byte for byte and widened the schedule to six areas: the three above, two further drawn environments (`drawn11`, twelve groups with four per world and no slip; `drawn13`, six groups with two per world and slip 0.15) and a second metric weighting (`metric555001`), none of the three lived by any seed before, in 18 lives (Table 5). Version 6 passed growth, safety and replay on both seeds, every one of the twelve area scores and eighteen pools rising, and failed cost on both, at 2.78 and 2.64 times living: a shared-language candidate that passes its own area is then checked on every other lived area, so the test bill grows with the number of areas, by 0.18 per area over version 4's mean. Version 7 changed only the cost threshold, to 2.5 plus 0.25 per area beyond three (3.25 at six), the 0.25 being the structural ceiling of one 24-world check against a 96-world life per candidate; it passed all four gates on both seeds, at 2.88 and 2.32 times living. On two of the four six-area seeds `metric555001` kept no change in its own two lives and still rose by 415% and 542% on its fresh set, from the language and models the other areas had built. Version 8 added the string-task area (`str7`), a second task form over a catalog of its own: version 7's schedule plus three string lives, 21 lives in all, with the cost threshold at 3.5. It passed all four gates on both seeds: fourteen of fourteen area scores rose, by 27% to 356% with a median of 78%, 22 of 22 pools rose, 24 of 24 acceptances replayed, and testing cost 2.61 and 2.71 times living, inside the six-area range, because a form whose catalog is disjoint adds its own tests and no cross-form check. The string area rose 27% and 32%, and both seeds kept the same six withheld positions, the end of the first, second and third upper-case run and the start of the first, second and third lower-case run, proposed from what stuck worlds had earned and accepted on held-out tasks with no label read; one seed also kept two positions outside the generator's universe. Forecasts were committed before each version ran; the probability given to

Table 4: Confirmation runs on sealed seeds: tasks solved on fresh held-out sets (24 worlds per pool; 7,200, 2,400 and 4,800 tasks) by the starting and the retained state, the test-to-live cost ratio, and the gate outcome. Version 4 is the confirmed protocol for three areas; version 5 added a guard and was retired; versions 6 to 8 ran six and seven areas (Table 5).

Seed (protocol)	mech	drawn7	metric	cost	gates
919100 (v1)	1,128 to 1,997	236 to 385	405 to 1,055	5.96×	cost fails
919101 (v1)	1,223 to 1,944	271 to 318	318 to 1,619	5.88×	cost fails
919102 (v2)	1,020 to 2,010	281 to 409	364 to 1,550	1.89×	own-set clause fails
919103 (v2)	1,200 to 1,697	255 to 363	363 to 1,428	1.98×	all pass
919104 (v3)	1,276 to 1,807	280 to 364	373 to 1,927	2.29×	one stale baseline
919105 (v3)	1,147 to 2,366	180 to 342	322 to 1,518	1.94×	one stale baseline
919106 (v4)	1,129 to 2,136	202 to 299	343 to 1,647	2.32×	all pass
919107 (v4)	1,138 to 2,263	275 to 382	248 to 1,901	2.04×	all pass
919108 (v5)	1,179 to 1,990	367 to 388	281 to 1,872	2.48×	growth and safety fail
919109 (v5)	1,111 to 2,151	278 to 334	404 to 1,626	2.53×	cost fails

Table 5: Six- and seven-area confirmation runs (versions 6 to 8; version 8 adds the string-task area and is otherwise version 4's code): tasks solved on fresh held-out sets (24 worlds per pool) by the starting and the retained state in each area, the test-to-live cost ratio, and the gate outcome. Version 6's cost gate was 2.5 times living; version 7's is 3.25 at six areas and version 8's 3.5 at seven.

Seed (protocol)	mech	drawn7	metric777001	drawn11	drawn13	metric555001	str7	cost, gates
919110 (v6)	1,197 to 2,442	305 to 375	299 to 1,415	291 to 424	375 to 483	356 to 1,833	–	2.78×, cost fails
919111 (v6)	1,187 to 2,160	305 to 384	355 to 1,585	206 to 341	176 to 314	283 to 1,475	–	2.64×, cost fails
919112 (v7)	1,014 to 2,070	214 to 374	493 to 1,723	217 to 415	237 to 375	308 to 1,390	–	2.88×, all pass
919113 (v7)	1,269 to 1,878	230 to 334	389 to 1,667	249 to 369	258 to 375	284 to 1,823	–	2.32×, all pass
919114 (v8)	1,185 to 2,300	166 to 460	344 to 1,422	249 to 381	215 to 439	371 to 1,605	4,749 to 6,026	2.61×, all pass
919115 (v8)	1,174 to 1,885	320 to 413	309 to 1,409	251 to 386	322 to 430	300 to 1,321	4,119 to 5,419	2.71×, all pass

all four gates passing on both seeds was 0.15, 0.35, 0.4, 0.3, 0.15, 0.2, 0.35 and 0.3. eftab:growth and a replay of every accepted change. The gates were growth of at least 15% in every area, no pool of any area below 90% of its start, testing at most 2.5 times the cost of living, and every accepted change replaying exactly. Table eftab:confirm gives the results. Growth, safety and replay passed on both seeds; every pool rose, the untrained combinations included (21.5 to 35.6 and 21.0 to 29.8 tasks per world). Cost failed on both, at 5.96 and 5.88 times living: the larger held-out sets adopted after the `s3` case, and a language candidate on nearly every life tested together and alone across every other area, multiply the test runs. The cross-area check rejected changes four times across the two seeds, for losses of 4% to 7% on the mechanism area that the proposing area's own test had accepted. The forecast gave the three passing gates 0.6, 0.45 and 0.8 and the failing gate 0.5.

7 Interference across areas

A change can improve the area that proposes it while damaging an area the agent has already learned. The first three-area check exposed exactly this failure.

After four lives in metric worlds, the main agent's score on the first drawn partition's untouched held-out set fell from 376 to 338, a 10% loss in an area it had not entered since. The cause was the shared language. The metric lives had accepted one extension of 40 atoms and one removal of 25, each tested on the metric area alone, and a language that grew and shrank to suit metric worlds changed the catalog the drawn partition's model had been fitted over. Each change was an

Table 6: Ablation of `s1`'s retained state (version 24) on the mechanism area's untouched held-out set: tasks per world by pool, and total of 7,200.

State	Trained	Unfamiliar	Untrained	Total
retained	49.5	18.7	18.2	2,072
retained, no prior	43.0	14.7	9.5	1,614
retained, no model	37.3	15.0	15.9	1,639
grown language only, no model, no prior	11.1	9.0	10.5	735
starting language, retained model and prior	50.0	6.1	26.2	1,973
starting state	26.2	3.7	24.2	1,299

accepted success where it was tested and a regression where it was not.

The revision was to the acceptance rule, made by the experimenter at version 23: a candidate that changes the language is also run on a held-out set of every other area the agent has lived in, 12 worlds per pool, and is rejected if its total there falls more than 2% below the current state's. The current scores of those areas are recomputed after any accepted shared change. The tolerance permits a small measured regression; it is a bound, not preservation. The cost is more held-out lifetimes per language candidate, which is why the combination of all proposals is tested first and the parts only if it fails.

The revised rule fired twice as soon as it existed. In `drawn11` a combination of invented atoms and a model gained 165 to 344 on that area's held-out set and was rejected because it cost mechanism worlds 8% and `drawn7` 12%; the model alone was then accepted, at 417. In `drawn13` a combination worth 150 was rejected for costing `drawn11` 23%. Each would have been accepted under the earlier rule. The `s3` case of Section 6 shows the rule's remaining gap: a 12-world check read a gain where a 24-world set read a 12% loss.

8 The grown language and the pools the agent cannot see

The five-area check at version 24 showed, on the mechanism area's untouched set, that untrained family combinations had fallen from 24 to 18 tasks per world while trained worlds rose from 26 to 50. The acceptance tests had not seen it, because a test compares totals and the untrained pool is a third of the set.

An ablation of the retained state on that set attributes the loss (Table 6). The retained model and prior on the starting language score 1,973 with untrained combinations at 26 per world, above the start; the grown language alone, with no model and no prior, scores 735 against 1,299 for the starting language; the retained state scores 2,072 with untrained at 18. The 166 invented atoms then in the language, 110 of them behaviourally new, had enlarged the catalog from 82,758 rows to 266,272, and the half of each task's budget that the prior rule reserves for the no-prior order was scanning a catalog two thirds of which the agent had added. Trained worlds did not suffer, because the model's half of the budget found their rows anyway; untrained combinations, which the model promotes less, did.

The agent already held the verdict it needed. Its self model files an atom in no group when the agent would almost never find it in a world. Across the five area models, 107 invented atoms were rated real by at least one model, 55 of the 56 that reproduce a withheld family among them, and 59 were rated import-only by every model that had seen them, one of them real. From version 26 the pruning operation proposes those too, and the acceptance test decides.

Two further revisions went in at the same version, both to the test. The interval requirement replaced the strict-inequality rule after the life-10 case. And the tie-break, the order the scan uses among rows it scores equally, had been a random permutation of row indices, so adding or removing any atom reshuffled every other row and every language test carried that noise; it is now a hash of each row's content, so a language change changes nothing but the rows it adds or removes.

On life 11 the removal of the 55 atoms was accepted, +0.3 to +3.0 tasks per world on the mechanism set and better in all four other areas, and the model and extension proposed beside it were rejected on intervals that crossed zero. The language went from 426 to 371 atoms. On the untouched set untrained combinations now read 18.6 against a start of 20.5, inside the 10% that the draft confirmation protocol allows, with every other pool and area above its start.

9 What this does and does not show

It shows one persistent agent improving its own state in five areas of different structure, and two further agent instances improving in three areas and two, keeping only changes it tested on worlds it never trained on, with the tests charged. The three forms of growth from the antecedents appear together in its ledger: within-life accumulation in every life, learned search guidance accepted nearly every life and accounting for most of the measured gain, and invented primitives that let it reach families its starting language could not express. Growth in one area did not require starting over in another; the language carried, and in the metric area a model fitted from 48 worlds was worth 975 tasks at once.

The development results of Table 3 were inspected as the design changed, on random seeds used throughout. The confirmation runs (Table 4) are the prespecified evidence: a frozen design, pre-stated gates, a committed forecast, sixteen sealed seeds in eight versions, and a replay of every accepted change. The cost gate failed in the first version; the audit found one bookkeeping flaw in each of the second and third versions, each fixed before the next pair of seeds; the fourth version passed all four gates on both seeds; the fifth added a guard, failed growth and safety on one seed and cost on the other, and was retired; the sixth and seventh widened the schedule to six areas, three of them new to every seed, and the seventh passed all four gates on both seeds; the eighth added a seventh area of a second task form and passed all four on both seeds. No replay overturned an acceptance. Growth of at least 15% in every area held on fifteen seeds of sixteen, and no pool below 90% of its start on fifteen of sixteen; the exceptions are both on the same seed, under the retired version. Across the six seeds of six or more areas, 38 of 38 area scores rose, by 23% to 542%, and 58 of 58 pools rose. The held-out sets behind both tables are single draws whose per-world results were not saved, so no interval accompanies the totals. Two accepted changes of a development seed's chain run do not replay, for a reason not found; the auditor replayed the 131 accepted changes of the sixteen confirmation seeds and found the two stale baselines above and nothing else.

It is self-improvement of state, not of procedure. The agent's model and language direct its search, its search produces the experience the next model and language are fitted from, and the ledger shows the state improving the state across twelve lives. But the procedures that turn experience into a model, candidates into atoms, and tests into acceptances are fixed code, and the two revisions of the acceptance rule were made by the experimenter. A later agent does not produce a better reflect or a better extend; the programme's definition of recursive self-improvement is not met.

The growth also saturates and is uneven. On the main agent's later mechanism lives few candidates passed, and model gains fell from hundreds to single digits by the sixth life. Untrained combinations in mechanism worlds ended 9% below their start. Half of the remaining invented atoms are behaviourally new and some are surely junk that costs budget until a removal takes them

out. Testing cost two and a half times living. And every area shares one task form and one catalog; the agent has not met a second domain.

10 Lessons

1. The acceptance test is the design. Every choice that mattered concerned what counts as an accepted change: a paired interval above zero on a fresh held-out set, within tolerance everywhere the state is shared, cost charged. Given that rule, a weak candidate costs a test and nothing else, so the agent can propose freely.

2. Test the combination first, then the parts. This halved the tests in the common case and lost nothing.

3. Invention needs recurrence. An atom that fits one uncovered task is usually a coincidence; one that fits two is usually a missing atom. The first version accepted single fits, and every extension it proposed lost.

4. Let the agent remove what it added. Removing atoms that never earned their keep was accepted three times in three, and removing the atoms the self model called import-only recovered a loss no test had seen.

5. A model from 48 worlds can be worth keeping, when a test decides.

6. Shared state needs shared tests. The 10% loss in an untouched area was invisible to any single-area test.

7. Score growth on worlds the agent never saw, and attach an interval. A gain of 24 on 72 worlds is noise.

8. Make the measurement stable under the change being measured. A tie-break over row indices turned every language change into a reshuffle of the whole catalog.

9. Build for the host. One stall and one crash in the record were the machine's shared memory, not the agent.

11 Next

- A guard for the untrained pool that bounds cumulative drift against the starting language's unguided search, since a per-candidate tolerance let an 11% dip accrue on one seed; a second task form; and, as the protocol requires, a further pair of sealed seeds for any change.

- Intervals on the reported gains: save per-world results in the final checks.

- Testing cost against test power: the half-set early stop is in, but the 12-world cross-area sets let a 12% loss through and the 24-world sets of single-pool areas rarely admit a change under the interval rule; both need more worlds, which costs more, so the cost cut must come from stopping early, not from smaller sets.

- A label-free guard for pools the agent cannot see, on the no-prior half of the budget, if the untrained-combination loss returns.

- More areas and a second domain: a second task form and a second alphabet in the same catalog, then the string-manipulation tasks, which require the agent's solver to be one of several.
- The next level: the agent improves its state, not its procedures. The first procedure it could plausibly improve is the acceptance test itself, learning from its ledger how large a held-out set each kind of candidate needs.

12 Reproducibility

The repository is the RSI project at `projects/rsi`. The seed agent is `rsi/seed.py` (state, areas, the four operations, the acceptance test) and `rsi/seed_string.py` (the string-task form) with `rsi/seed_areas.py` for metric worlds, `rsi/invention.py` for the grammar and the recurrence test, and `rsi/self_world_model.py` for the model. `seed_run.py--names1--seed644101--areamech--lives4` runs lives and resumes from the saved state; `seed_check.py--names1--areasmech,drawn7,metric777001` scores an agent on held-out sets no test used and labels its inventions; `seed_ablate.py` produced Table 6; `audit_seed.py` replays accepted tests from version snapshots; `confirm_seed.sh<name><seed>` runs the frozen confirmation schedule, check and audit of `EXPERIMENT-seed-confirmation.md`, whose sixteen runs are in `seeds/c1` to `seeds/c16`, four over six areas and the last two over seven in two task forms. The worked task of Section 2 is reproduced by drawing the first trained world of `MechanismArea(644101)` with `random.Random(7)` and the task from its first mechanism with `random.Random(11)`. States are in `seeds/<name>/state.json`, committed after every run; the ledger inside each is the full record of every candidate, its test and its cost. Logs are `runs_seed_*.log` and the checks `analysis/seed-check-<name>.json`.

Settings: 48 worlds of 100 tasks per life; budget 1,024 scans per task; 24 worlds per pool for an area's held-out set and 12 for the cross-area check; tolerance 2% relative; group counts 6 to 8 and 2 to 4 groups per world by held-out likelihood; extension cap 40 atoms per life with recurrence 2; removal after 2 lives or on the self model's import-only verdict; tie-break a content hash per row with seed 77; a change accepted only when the paired 95% bootstrap interval (4,000 draws) of its gain is above zero. Area specifications are stored in the state so that a test can rebuild any area the agent has lived in. `RSI_WORKERS` caps the worker pool; six or fewer is safe on this machine beside other jobs.

Agents: `s1` on random seed 644101 (versions 1 to 26 reported here), `s2` on 643101 and `s3` on 645101. The companion paper on the world and self model is `SELF-SIMULATION-WORLD-MODEL-2026-10-10.md`, the distillation of earlier growth results is `GROWTH-DISTILLATION-2026-10-10.md`, and a living copy of this paper is at `https://claude.ai/code/artifact/ebd48ba7-84b1-4bb2-b2e5-eb18375651bc`.